

Artificial Intelligence Assignment 5

By Himanshu Sardaan

Question 1

Solve the given 0/1 Knapsack problem by considering the following points:

Name	Weight	Value
A	45	3
B	40	5
C	50	8
D	90	10

Chromosome is a 4 bit string $\{x_a, x_b, x_c, x_d\}$

Population size = 4, Maximum Capacity of the bag (W) = 100

First two fittest chromosomes selected as it is. 3rd and 4th fittest are used for one point crossover in the middle followed by single bit mutation of the first offspring

Bits chosen for mutation follows this cyclic order $\{x_D, x_C, x_B, x_A\}$

Initial Population: {1111, 1000, 1010, 1001}

Output the result after 10 iterations.

```
items = [  
    {"name": "A", "weight": 45, "value": 3},  
    {"name": "B", "weight": 40, "value": 5},  
    {"name": "C", "weight": 50, "value": 8},  
    {"name": "D", "weight": 90, "value": 10},  
]  
  
max_weight = 100  
population = ["1111", "1000", "1010", "1001"]  
mutation_order = [3, 2, 1, 0]  
  
def fitness(chromosome):  
    total_weight = total_value = 0  
    for i, bit in enumerate(chromosome):
```

```

        if bit == "1":
            total_weight += items[i]["weight"]
            total_value += items[i]["value"]
        return total_value if total_weight ≤ max_weight else 0

def crossover(parent1, parent2):
    point = len(parent1) // 2
    return parent1[:point] + parent2[point:], parent2[:point] +
parent1[point:]

def mutate(chromosome, iteration):
    idx = mutation_order[iteration % len(mutation_order)]
    chromosome = list(chromosome)
    chromosome[idx] = "0" if chromosome[idx] == "1" else "1"
    return "".join(chromosome)

for iteration in range(10):
    sorted_population = sorted(population, key=fitness, reverse=True)
    new_population = sorted_population[:2]

    offspring1, offspring2 = crossover(sorted_population[2],
sorted_population[3])
    offspring1 = mutate(offspring1, iteration)
    population = new_population + [offspring1, offspring2]
    best = max(population, key=fitness)
    print(f"Iteration {iteration + 1}: Best Chromosome: {best}, Fitness:
{fitness(best)}")

best_solution = max(population, key=fitness)
print(f"\nFinal Best Solution: {best_solution}, Fitness:
{fitness(best_solution)}")

```

Question 2

A thief enters a house for robbing it. He can carry a maximal weight of 9kg into his bag. There are 4 items in the house with the following weights and values. The thief has to plan the items he should take to maximize the total value if he either takes the item completely or leaves it completely?

Item	Item Name	Weight	Value
A	Mirror	2	3
B	Silver Nugget	3	5
C	Painting	4	7
D	Vase	5	9

The problem is solved using Genetic Algorithm with population size 4 and each individual encoded as $\{X_A, X_B, X_C, X_D\}$ where $X_i = \{0, 1\}$ and $i = A, B, C, D$.

Consider initial population as 1111, 1000, 1010, and 1001.

Generate the population for next iteration as follows: Select the 1st and 2nd fittest individual as it is in the next iteration. Apply 1-point crossover in the middle between 3rd and 4th fittest chromosome followed by single bit mutation of first offspring (produced through crossover).

Bit chosen for mutation follows this cyclic order $\{X_C, X_A, X_D, X_B\}$

Output the result after four iterations.

```
items = [
    {"name": "Mirror", "weight": 2, "value": 3},
    {"name": "Silver Nugget", "weight": 3, "value": 5},
    {"name": "Painting", "weight": 4, "value": 7},
    {"name": "Vase", "weight": 5, "value": 9},
]
max_weight = 9

population = ["1111", "1000", "1010", "1001"]

def fitness(chromosome):
    total_weight = total_value = 0
    for i, bit in enumerate(chromosome):
        if bit == "1":
            total_weight += items[i]["weight"]
            total_value += items[i]["value"]
    return total_value if total_weight ≤ max_weight else 0

def crossover(parent1, parent2):
    point = len(parent1) // 2
    return parent1[:point] + parent2[point:], parent2[:point] +
parent1[point:]

def mutate(chromosome, iteration):
    idx = mutation_order[iteration % len(mutation_order)]
    chromosome = list(chromosome)
    chromosome[idx] = "0" if chromosome[idx] == "1" else "1"
    return "".join(chromosome)

mutation_order = [2, 0, 3, 1]

for iteration in range(4):
    fitness_values = [(chrom, fitness(chrom)) for chrom in population]
    fitness_values.sort(key=lambda x: x[1], reverse=True)
```

```

new_population = [fitness_values[0][0], fitness_values[1][0]]
child1, child2 = crossover(fitness_values[2][0], fitness_values[3][0])
new_population.extend([child1, child2])

mutation_index = mutation_order[iteration % len(mutation_order)]
new_population[2] = mutate(new_population[2], mutation_index)
population = new_population

best_solution = max(population, key=fitness)
best_items = [items[i]["name"] for i in range(4) if best_solution[i] == "1"]
best_value = fitness(best_solution)
best_weight = sum(items[i]["weight"] for i in range(4) if best_solution[i] == "1")

print("Best Selection (Binary):", best_solution)
print("Selected Items:", best_items)
print("Total Weight:", best_weight)
print("Total Value:", best_value)

```

Question 3

Consider the following 2-SAT problem with 4 Boolean variables a, b, c, d:

$$F = (\neg a \vee d) \wedge (c \vee b) \wedge (\neg c \vee \neg d) \wedge (\neg d \vee \neg b) \wedge (\neg a \vee \neg d)$$

The MOVEGEN function to generate new solution be arbitrary changing value of any one variable

Let the candidate solution be of the order (abcd) and the initial candidate solution be (1111).

Let heuristic to evaluate each solution be number of clauses satisfied in the formula.

Apply Simulated Annealing (Consider $T = 500$ and cooling function = $T - 50$)

(Assume the following 3 random numbers: 0.655, 0.254, 0.432)

Accept every good move and accept a bad move if probability is greater than 50%.

```

import random
import math

def evaluate(solution):
    a, b, c, d = solution
    clauses = [
        (not a or d), # (¬a ∨ d)
        (c or b),      # (c ∨ b)
        (not c or not d), # (¬c ∨ ¬d)
        (not d or not b), # (¬d ∨ ¬b)
    ]

```

```

        (not a or not d)    # ( $\neg a \vee \neg d$ )
    ]
    return sum(clauses)

def movegen(solution):
    new_solution = solution[:]
    index = random.randint(0, 3) # Choose a random index
    new_solution[index] = 1 - new_solution[index] # Flip the bit
    return new_solution

def simulated_annealing():
    current_solution = [1, 1, 1, 1]
    best_solution = current_solution[:]
    best_score = evaluate(best_solution)

    T = 500
    cooling_rate = 50
    random_numbers = [0.655, 0.254, 0.432]

    for i in range(3):
        new_solution = movegen(current_solution)
        new_score = evaluate(new_solution)
        current_score = evaluate(current_solution)

        if new_score > current_score:
            current_solution = new_solution
        else:
            probability = math.exp((new_score - current_score) / T)
            if random_numbers[i] < probability:
                current_solution = new_solution

        if evaluate(current_solution) > best_score:
            best_solution = current_solution[:]
            best_score = evaluate(best_solution)

        T -= cooling_rate

    return best_solution, best_score

best_solution, best_score = simulated_annealing()

print(f"Best Solution: {best_solution} (a={best_solution[0]}, b={best_solution[1]}, c={best_solution[2]}, d={best_solution[3]})")
print(f"Max Clauses Satisfied: {best_score}")

```